

HireMap : Term 1 Summary

Asmita Singh

Live system: <https://hiremap-dashboard.onrender.com>

Before searching, please wake the backend (it sleeps when idle on free-tier hosting):

1. Open <https://hiremap-ffey.onrender.com/api/health> and wait until it displays `{"status": "ok"}` (may take 30–50 seconds the first time).
2. Open <https://hiremap-worker.onrender.com/> and wait until it displays: `{"worker alive"}`.
3. Once both respond, open the dashboard above and run a search. Results will appear within a few seconds.

Source code: <https://github.com/asmitasingh0323/HireMap>

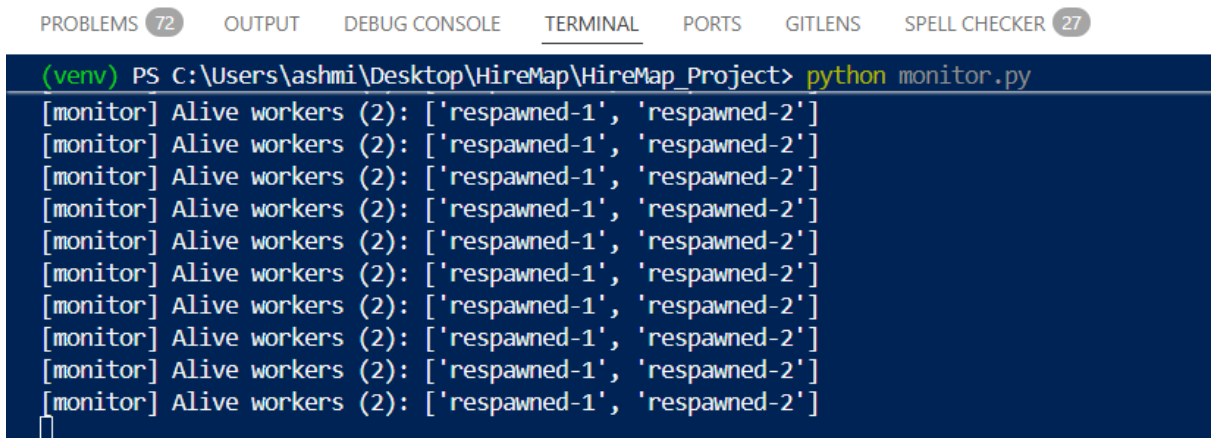
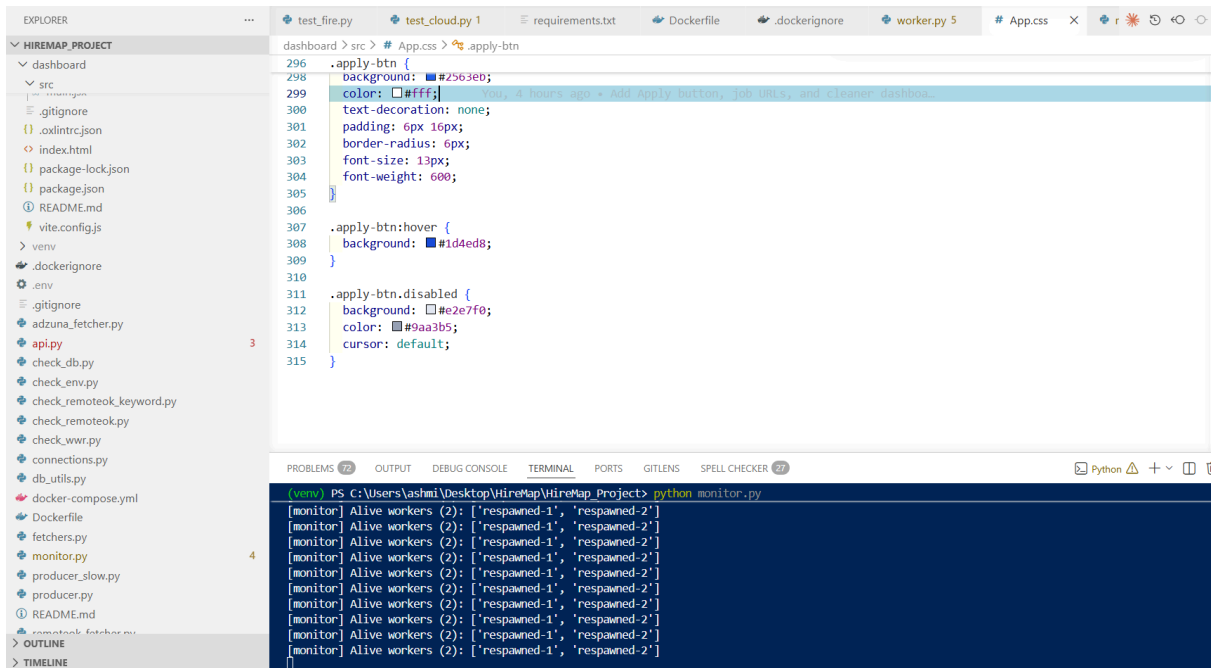
1. Overview

HireMap is a fault-tolerant distributed pipeline that aggregates job postings from multiple external sources in parallel, normalizes them into a single schema, deduplicates across sources, and returns results within a user-specified deadline. It is a distributed-systems project that uses job-market data as its workload. The job aggregation is the concrete, real-world problem the system is exercised against, but the important thing is the systems engineering underneath it: coordinating independent processes through a message broker, guaranteeing correctness under concurrency, recovering from failure, and enforcing a response-time bound. The same architecture would apply equally to aggregating weather, pricing, or any other fragmented, time-sensitive data. Over the first term, it progressed from an architectural proposal into a fully working, cloud-deployed system that can be accessed at a public URL.

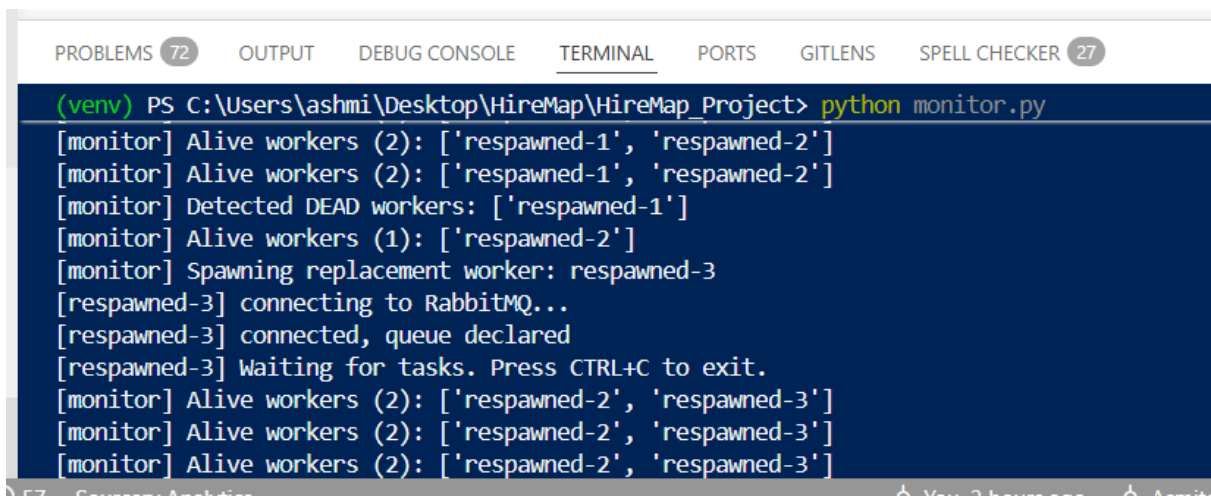
2. What Was Built

The system was implemented in full, following the architecture defined in the original proposal.

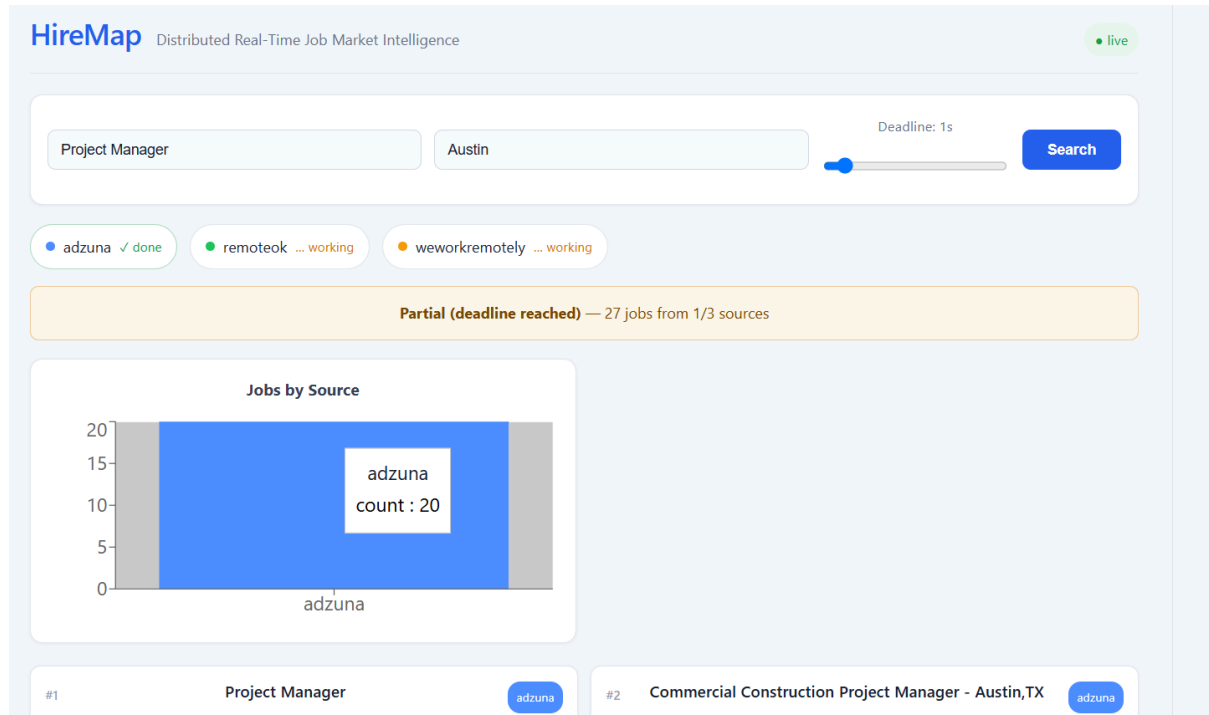
Distributed worker-queue model: A Flask API accepts a search (keyword, location, deadline) and publishes one task per data source to a RabbitMQ message broker. Independent, stateless worker processes consume these tasks in parallel. Workers share no memory; all coordination flows through the broker. This realizes the producer–consumer pattern that is the core of the proposal.



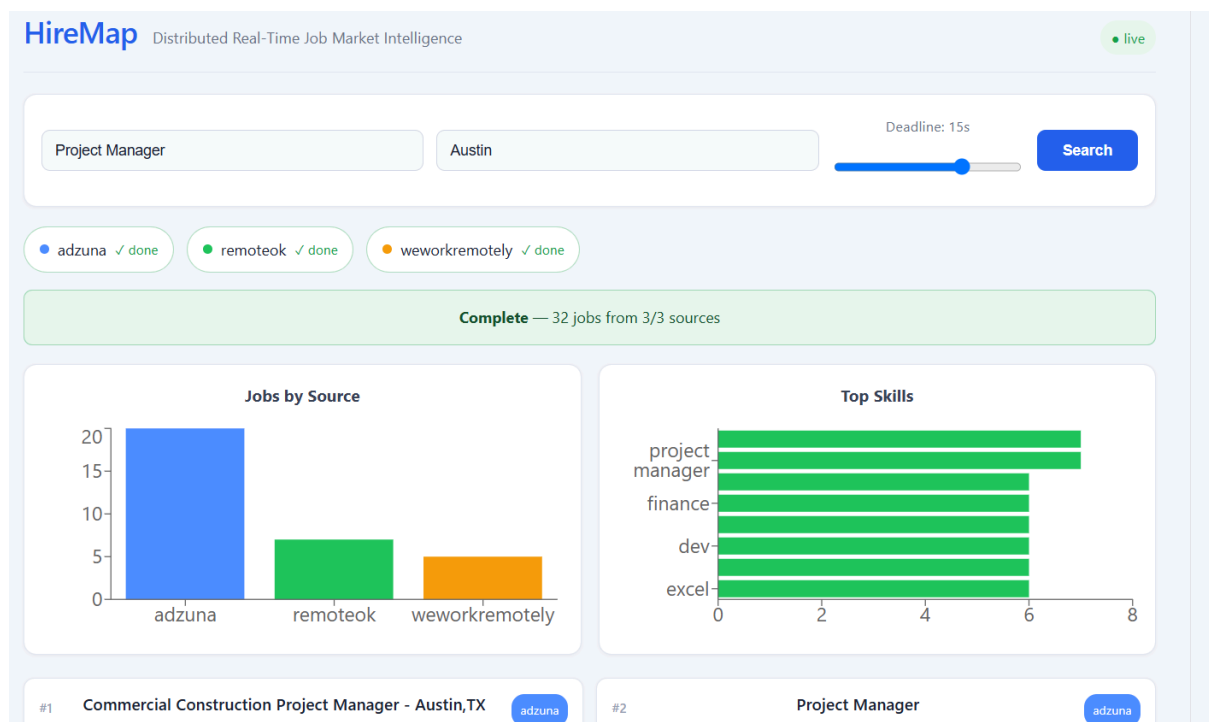
Forcefully killed a worker making sure another worker comes to live.



SLA-bounded execution: The API enforces the user's deadline. When the deadline is reached, it returns whatever results have arrived rather than waiting for slow sources, implementing the proposal's deliberate availability-over-completeness tradeoff. Completion is tracked in the database as the single source of truth, distinguishing "a source finished" from "a source returned data."

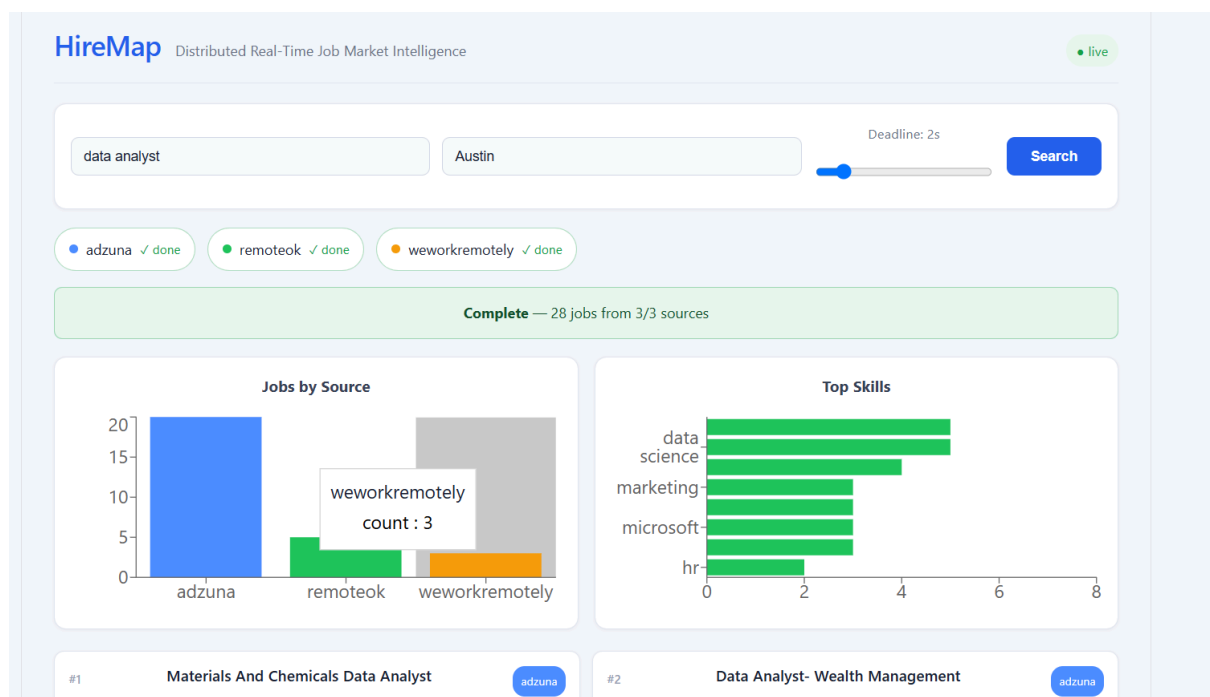
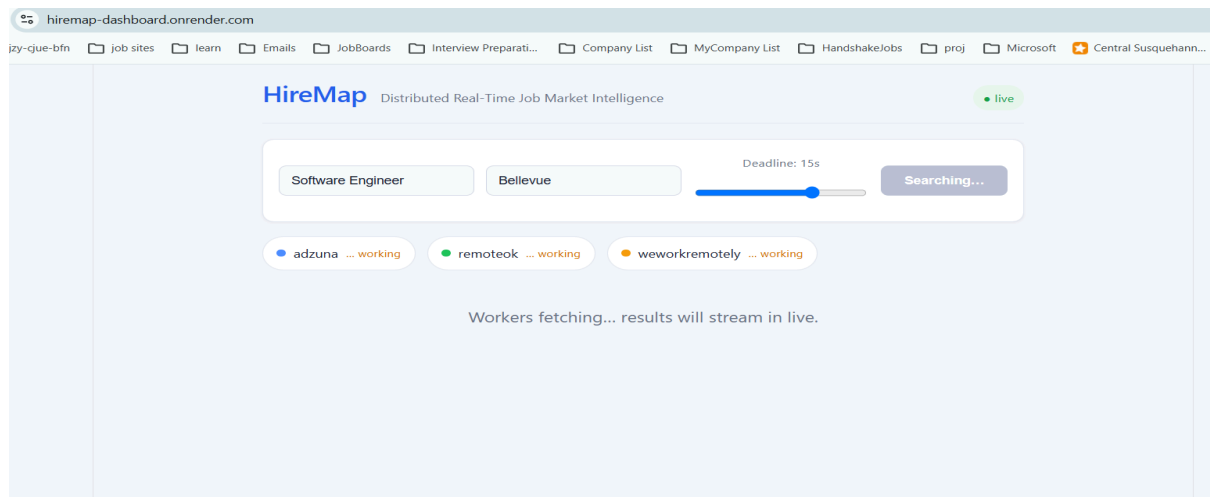


Here in the above screenshot I am searching for a fresh keyword with the deadline slider at **1 second**. It shows **Partial (deadline reached)**.



Then I again ran the same search in the above screenshot with the deadline at **15 seconds** it shows: **Complete 32 jobs from 3/3 sources.**

Real-time dashboard: A React + Recharts frontend connects to the API over WebSockets. Results stream into the interface the moment each source's worker finishes, alongside a live per-source status panel and charts summarizing jobs by source and top skills.



#1 Materials And Chemicals Data Analyst TEKsystems c/o Allegis Group Bee Cave, Travis County · \$105,573+ Apply → adzuna	#2 Data Analyst- Wealth Management TUPPL Technology Inc Bee Cave, Travis County · \$170,800+ Apply → adzuna
#3 Data Analyst SafeLease Austin, Travis County · \$129,442+ Apply → adzuna	#4 Senior Data Analyst Oracle Tarrytown, Travis County · \$108,682+ Apply → adzuna
#5 Data Analyst II ONE Gas Tarrytown, Travis County · \$65,329+ Apply → adzuna	#6 Data Analyst II (remote) Ryder System Tarrytown, Travis County · \$93,027+ Apply → adzuna
#7 Intern Conversion: Data Analyst IBM Tarrytown, Travis County · \$70,368+ Apply → adzuna	#8 Data Analyst-Technical-Staff Intermountain Health Tarrytown, Travis County · \$116,095+ Apply → adzuna

3. Evaluation Against the Three Criteria

The proposal defined three criteria by which the system should be judged. All three were demonstrated on the running system.

Correctness & Idempotency: Running the distributed pipeline produces the same deduplicated result set as a sequential single-threaded run of the same sources. Under concurrent parallel writes, overlapping jobs were correctly caught and skipped (observed directly as non-zero "skipped duplicate" counts in worker logs), confirming the fingerprinting layer holds under race conditions.

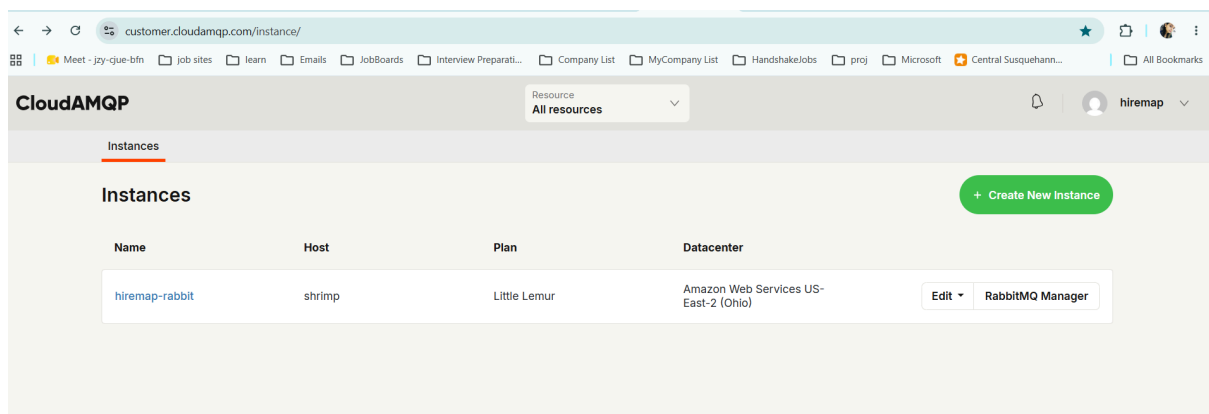
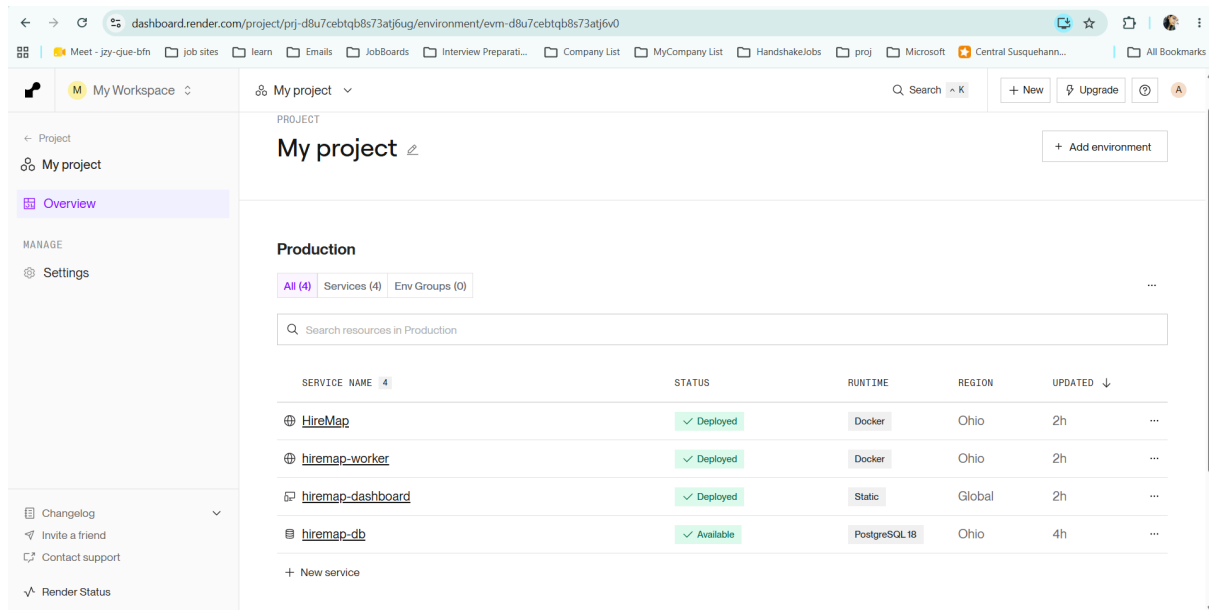
Fault Recovery: A worker was violently terminated mid-task. Because it had not yet acknowledged its message, RabbitMQ automatically requeued the task, and a surviving worker picked it up and completed it with no task lost and no duplicate created. Separately, the Failure Monitor detected the dead worker (missed heartbeats) and spawned a replacement, restoring the pool to its minimum size(*screenshot attached above*). Fault tolerance therefore operates at two independent levels: task-level (broker requeue) and process-level (monitor respawn).

Deadline Adherence: The same search was run twice. Under a 1-second deadline, the system returned partial results and correctly reported the search as incomplete. Under a generous deadline, the same search completed with all sources finished. The deadline is enforced as a hard bound, not a best-effort target, and the system returns valid partial results without crashing regardless of remaining work.

4. Deployment

Beyond the local Docker build, the complete system was deployed to the cloud as five coordinated services:

- PostgreSQL (Render),
- RabbitMQ (CloudAMQP),
- the API,
- the worker, and
- the dashboard.



This required refactoring the codebase to be fully environment-driven, reading database and broker connection details from environment variables rather than assuming a local machine, so the same code runs unchanged both locally and in the cloud. The result is a genuinely live, shareable system rather than a local-only demonstration.

5. Challenges and What I Learned

Building and deploying the system surfaced a series of real obstacles, each of which taught a lot of lessons.

Early on, locally installed PostgreSQL and RabbitMQ instances silently intercepted connections on their default ports, producing authentication failures that looked like credential bugs. Resolving this clarified how services bind to ports and why connection configuration should never assume "localhost."

The most instructive bug was in deduplication: an initial approach that checked for a fingerprint before inserting worked sequentially but allowed duplicates once workers ran in parallel. Replacing it with a database UNIQUE constraint and ON CONFLICT taught me directly why check-then-act is unsafe under concurrency and why pushing the guarantee down to the database is the correct, idempotent solution.

Cloud deployment exposed the gap between "works on my machine" and "works in production." A worker appeared dead in its logs but was actually running: its output was simply being buffered and never flushed. The API crashed silently because PostgreSQL date and Decimal values cannot be serialized to JSON without conversion. And a search returned nothing because a database column added locally for a later feature was never added to the cloud database which was a schema-drift problem. Each of these was invisible until traced through the logs, and together they taught me how much of distributed-systems work is disciplined observation and debugging rather than writing new code.

6. Future Work

The first term established a reliable distributed-systems foundation. A natural second term would build intelligence on top of it. The current keyword matching is literal string comparison, which misses semantically related roles, a limitation observed directly during testing. The strongest extension is embedding-based semantic search: a new class of worker that converts jobs and queries into vector representations, backed by a vector store, so that a search for "ML engineer" also surfaces "machine learning specialist" and related titles. This keeps the project firmly in distributed systems (a new distributed worker type and a new storage tier) while adding genuine AI-driven capability. Secondary goals would include autoscaling the worker pool based on queue depth and adding proper metrics and observability. A detailed bi-weekly plan for this extension is provided separately.